NAQLA

LAB 4

Lab 4 — Shahna Assistant

Agentic Backend Engineering

Agentic Backend Engineering — Build Scalable Backends With and For AI Agents

Lab 4 — Shahna Assistant

Overview

Goal

Until today, the AI lived in your terminal. Today it moves *into* the backend. By the end of the lab, Shahna customers can chat with **Shahna Assistant** in Arabic or English through `POST /api/agent/chat`: ask for a price, book a shipment (only after they confirm), track or cancel a shipment, and ask policy questions that are answered from real company documents.

Today's habits:

1. **The LLM is an unreliable dependency.** Wrap it, give it a schema, give it narrow tools, and test it with fakes.
2. **The agent does not know Prism.** Your coding agent will confidently write Prism code from memory. Prism changes between versions. The source code in `vendor/prism-php/prism` is the truth.

What you will build

Piece	Where it lives
Model configuration	`config/shahna.php`
Intent classifier (Haiku, structured output)	`app/Agent/IntentClassifier.php`
Email-to-quote extractor (structured output)	`app/Agent/ShipmentEmailExtractor.php`
Policy documents with full-text search	`app/Models/PolicyDocument.php`, migration, seeder
Six agent tools	`app/Agent/Tools/`
The agent itself	`app/Agent/ShahnaAssistant.php`
Conversation memory	`AgentConversation`, `AgentMessage` models
Chat endpoint	`POST /api/agent/chat`
Tests using `Prism::fake()`	`tests/Feature/Agent/` (`IntentClassifierTest`, `ShipmentEmailExtractorTest`, `ToolsTest`, `AgentChatTest`)

Starting point

```
git checkout day3-end
git switch -c my-day4
sail up -d
sail artisan migrate:fresh --seed
sail artisan test
```

All 80 tests should pass before you start. If they do not, tell your instructor now, not at 15:30.

Add your Anthropic API key (provided by your instructor) to `.env`:

```
ANTHROPIC_API_KEY=sk-ant-...
SHAHNA_AGENT_MODEL=claude-sonnet-5-5
SHAHNA_FAST_MODEL=claude-haiku-5-5
```

WARNING

Never commit `.env`, never paste your key into a prompt, and never put it in a test. Check that `.env` is listed in `.gitignore` before your first commit today. Every test in this lab uses `Prism::fake()`; only your manual curl and Tinker calls spend real tokens.

Time budget

Part	Step	Minutes
Part 1 (11:00–12:30)	1. Check Prism and add the config, by hand	15
	2. First LLM call in Tinker, by hand	10
	3. IntentClassifier with structured output	25
	4. ShipmentEmailExtractor	20
	Checkpoint 1	20
Part 2 (14:15–15:45)	5. Policy documents and full-text search	25
	6. The six tools	25
	7. ShahnaAssistant and conversation memory	20
	8. The chat endpoint and manual tests	15

Part	Step	Minutes
	Checkpoint 2 and commit	5

Part 1 — Talking to an LLM from Laravel (11:00–12:30)

Step 1 — Check Prism and add the config (15 min)

What and why. Prism (`prism-php/prism`) is a Laravel package that gives one API over many LLM providers. We use its Anthropic provider. It has been in `composer.json` since `day1-start` (version `^0.102`), so there is nothing to install. You will check it and write the config file yourself so you know exactly where model names come from.

> **WARNING**
> **By hand.** Do this step without the agent.

```
sail composer show prism-php/prism | head -3
```

Open `vendor/prism-php/prism/config/prism.php` and find the `anthropic` block. Confirm it reads `env('ANTHROPIC_API_KEY')`. You do not need to publish or change that file.

Now create `config/shahna.php`:

```
<?php

return [
    'agent_model' => env('SHAHNA_AGENT_MODEL', 'claude-sonnet-5-5'),
    'fast_model' => env('SHAHNA_FAST_MODEL', 'claude-haiku-5-5'),

    // Upper limit on model round-trips (tool calls + final answer) for one chat message.
    'max_steps' => 6,

    // How many earlier messages of a conversation are sent back to the model.
    'history_limit' => 20,
];
```

Why a config file instead of `env()` in your classes? Because `php artisan config:cache` in production makes `env()` return `null` outside config files. Code reads `config('shahna.agent_model')`; only config files read `env()`.

```
sail artisan tinker --execute="dump(config('shahna'));"
```

```
array:4 [
  "agent_model" => "claude-sonnet-5-5"
  "fast_model" => "claude-haiku-5-5"
  "max_steps" => 6
  "history_limit" => 20
]
```

Add one line to `CLAUDE.md` so the agent stops guessing:

```
- Prism API: read vendor/prism-php/prism/src before using any Prism class or method. Do not rely on memory. LLM tests must use Prism::fake().
```

Step 2 — Your first LLM call, in Tinker (10 min)

What and why. Before any abstraction, see the raw thing: one request, one response, a token count, a cost.

> **WARNING**
>
> **By hand.** Type this yourself in Tinker. No agent.

```
sail artisan tinker
```

```
> use Prism\Prism\Facades\Prism;
> use Prism\Prism\Enums\Provider;
> $r = Prism::text()
    ->using(Provider::Anthropic, config('shahna.fast_model'))
    ->withSystemPrompt('You are a Libyan logistics assistant. Answer in one sentence.')
    ->withPrompt('What is the largest port city in eastern Libya?')
    ->asText();
> $r->text;
= "Benghazi is the largest port city in eastern Libya."
> $r->usage->promptTokens;
= 31
> $r->usage->completionTokens;
= 14
> $r->finishReason;
= Prism\Prism\Enums\FinishReason {#... +name: "Stop"}
```

Your exact text and token counts will differ. Run the same call two more times. Is the answer identical? It probably is not word-for-word. That is why we will never assert on free text in tests.

Now open `vendor/prism-php/prism/src/Text/Response.php` in your editor. Find the `usage`, `steps`, `toolCalls` and `messages` properties. You will need all four this afternoon, and now you know where they really live.

Step 3 — IntentClassifier with structured output (25 min)

What and why. Most customer messages need routing, not a full agent run: "is this a price question, a tracking question, or something else?" A cheap, fast model (Haiku) with **structured output** returns JSON that matches a schema, so your code gets an array, not prose to parse.

Target contract:

```
{"intent": "quote", "language": "ar"}
```

`intent` is one of `quote|book|track|cancel|policy|other`; `language` is `ar|en`.

AGENT PROMPT

Agent prompt

```
Plan first and wait for my approval. Small diffs.

Create App\Agent\IntentClassifier with a method
  classify(string $message): array{intent: string, language: string}
It must use Prism::structured() with the Anthropic provider and
config('shahna.fast_model'). Build the schema with Prism's ObjectSchema and
EnumSchema: intent in [quote, book, track, cancel, policy, other],
language in [ar, en], both required.

Before writing code, open vendor/prism-php/prism/src/Schema/ObjectSchema.php,
Schema/EnumSchema.php and Testing/StructuredResponseFake.php and quote the
constructor signatures you will use.

If the model returns an intent or language outside the enum, fall back to
['intent' => 'other', 'language' => 'en'] — never throw.

Write the Pest test first at tests/Feature/Agent/IntentClassifierTest.php
using Prism::fake() with StructuredResponseFake::make()->withStructured([...]).
Cover: a normal classification (assert the request used the fast model) and
an invalid enum value from the model (fallback). Run the tests red, then
implement, then green.
```

What to check in the agent's output.

- Good: it quotes the real constructor `new ObjectSchema(name:, description:, properties:, requiredFields:)` and `new EnumSchema(name:, description:, options:)` from the vendor files.
- Good: tests call `Prism::fake([...])` before calling the classifier, and the fake is returned from `Prism::fake()` so the test can assert on the request (for example with `$fake->assertRequest(fn ($requests) => ...)`).
- Good: the fallback validates against the enum options in PHP, not by trusting the model.
- Red flag: imports that do not exist, such as `Prism\Prism\Schema\EnumSchema::make()` or `Prism::structured()->withJsonSchema()`. Search for the method yourself: `grep -rn "function withSchema" vendor/prism-php/prism/src`. If grep finds nothing, the agent invented it.

- Red flag: a test that hits the real API (no `Prism::fake()`), or a test that reads `ANTHROPIC_API_KEY`.

```
sail artisan test --filter=IntentClassifier
```

```
   PASS  Tests\Feature\Agent\IntentClassifierTest
  ✓ it classifies a message with the fast model
  ✓ it falls back to a safe default when the model returns something off-schema

  Tests:    2 passed
```

(These are the reference test names. Yours may differ; you may also have more cases.)

Now try it for real (this spends a few tokens):

```
sail artisan tinker --execute="dump(app(App\Agent\IntentClassifier::class)->classify('وين شحنتي
        SHN-7K2Q9X؟'));"
```

```
array:2 [
  "intent" => "track"
  "language" => "ar"
]
```

Try five messages of your own, including a tricky one ("I want to cancel my quote request and just ask about insurance"). Write down any wrong classification; it becomes an eval case tomorrow.

Step 4 — ShipmentEmailExtractor (20 min)

What and why. Shahna's sales team receives emails such as *"Salam, we need to move about 2 tons of tiles from Misrata to Sabha next week, flatbed preferred."* The extractor turns that into a quote request your `PricingService` can use. The important lesson: **when the email does not say something, the answer is `null`, not a guess.**

Target contract:

```
{
  "origin_city_code": "MRA",
  "destination_city_code": "SEB",
  "vehicle_type": "flatbed",
  "weight_kg": 2000,
  "pickup_date": null,
  "notes": "Tiles"
}
```

AGENT PROMPT

Agent prompt

```
Plan first and wait for approval. Tests first. Small diff.

Create App\Agent\ShipmentEmailExtractor::extract(string $email): array
returning origin_city_code, destination_city_code, vehicle_type, weight_kg,
pickup_date (YYYY-MM-DD) and notes.
- Use Prism::structured() with config('shahna.fast_model').
- Schema: ObjectSchema with StringSchema / NumberSchema properties, every
  one nullable: true and every one listed in requiredFields.
- System prompt: list the city codes (from the cities table) and the
  App\Enums\VehicleType values; use null for anything not stated in the
  email; never guess.
- After the model responds, validate in PHP with Validator, exactly like
  user input: city codes and vehicle type must be real values (Rule::in),
  weight_kg 1..25000, pickup_date date_format:Y-m-d. Invalid values throw
  a ValidationException. Return weight_kg as an int.

Tests in tests/Feature/Agent/ShipmentEmailExtractorTest.php using
StructuredResponseFake: a complete email, an email with missing fields
(expects null, not a guess), and a fake response containing an invalid
city code "XYZ" (expects a ValidationException).
Also assert the system prompt sent to the model contains the word "null".
```

What to check.

- Good: the allowed vehicle types come from `array_column(VehicleType::cases(), 'value')` and the city codes from the `cities` table, not a hard-coded copy.
- Good: PHP-side validation after the model call. Structured output makes valid JSON likely; it does not make *correct values* guaranteed.
- Red flag: the system prompt says “make a reasonable assumption if missing”. That is exactly the behaviour you do not want for a price.
- Red flag: the extractor creates a `Quote` itself. Its job is extraction only; pricing stays in `PricingService`.

```
sail artisan test --filter=ShipmentEmailExtractor
```

```
  Tests:    3 passed
```

CHECKPOINT

Checkpoint 1 (12:30)

```
sail artisan test
```

```
  Tests:    N passed
```

`N` is your Day 3 count (80) plus at least 5 new tests, with **zero failures** and no network calls (try it with Wi-Fi off: it must still pass).

Manual check with the real model:

```
sail artisan tinker --execute="dump(app(App\Agent\ShipmentEmailExtractor::class)
       ->extract('Hi, 800 kg of medicine from Tripoli to Benghazi, needs to stay
        cold.'));"
```

```
array:6 [
  "origin_city_code" => "TIP"
  "destination_city_code" => "BEN"
  "vehicle_type" => "reefer"
  "weight_kg" => 800
  "pickup_date" => null
  "notes" => "Medicine, must stay cold"
]
```

Note: "needs to stay cold" leading to `reefer` is an inference. Decide with your partner whether that is acceptable or whether it should be `null`. There is no single right answer; there should be a *written* answer in your system prompt.

```
git add -A
git commit -m "feat: prism, intent classifier, email extractor"
```

Part 2 — Tools, Retrieval and the Agent Loop (14:15–15:45)

Step 5 — Policy documents and full-text search (25 min)

What and why. The assistant must answer policy questions ("Can I ship gas cylinders?") from Shahna's real policies, not from the model's imagination. This is **retrieval**: find the relevant text, put it in the prompt. We use PostgreSQL full-text search, so there is no extra service and no extra API key.

First, the table. Ask the agent for the migration, model and seeder:

AGENT PROMPT

Agent prompt

```
Plan first and wait for approval. Small diff.

1. Migration for policy_documents: id, slug (unique), title, body (text),
   timestamps. Then, only when DB::getDriverName() === 'pgsql' (tests run
   on SQLite, which has no tsvector), add with DB::statement
   search_vector tsvector GENERATED ALWAYS AS (
     setweight(to_tsvector('english', coalesce(title, '')), 'A') ||
     setweight(to_tsvector('english', coalesce(body, '')), 'B')) STORED
   and a GIN index policy_documents_search_vector_index on search_vector.
   down() must drop the table.
2. App\Models\PolicyDocument with fillable slug, title, body.
   search_vector must NOT be fillable (Postgres computes it); add it to
   $hidden.
3. PolicyDocumentSeeder (updateOrCreate by slug) with 5 documents:
   cancellation, prohibited-goods, insurance, delivery-times, payment.
   Titles in English and Arabic ("Cancellation Policy — سياسة الإلغاء");
   bodies in English, 100-200 words each.
   Cancellation must match our ShipmentStatus rules: free in pending and
   confirmed, a fee once assigned, not possible after picked_up.
   Prices and payment are in LYD (cash to the driver at pickup or bank
   transfer).
4. Register it in DatabaseSeeder. Do not write the search code yet.
Run migrate:fresh --seed and show the output.
```

What to check.

- Good: the `'english'` text-search configuration with `setweight` (title matches rank above body matches). It stems English words, so “cancelled” finds “cancel”. It does not understand Arabic, which is why the `search_policies` tool asks the model for a few *English* keywords, even when the customer writes in Arabic.
- Good: the PostgreSQL-only SQL is guarded with `DB::getDriverName() === 'pgsql'`, as `CLAUDE.md` requires. Without the guard, every test fails on SQLite.
- Red flag: the seeded cancellation policy contradicts your enum (for example, “you can cancel while in transit”). Policy text and code must agree, or the agent will promise things the API refuses.
- Red flag: `search_vector` filled from PHP in a model observer. The generated column already does it.

Now write the search query **by hand** in `psql` before any PHP exists.

WARNING

By hand. Write and run this SQL yourself. Understanding the query is the point.

```
sail psql
```

```
SELECT slug,
       ts_rank(search_vector, websearch_to_tsquery('english', 'cancel or fee')) AS rank
FROM policy_documents
WHERE search_vector @@ websearch_to_tsquery('english', 'cancel or fee')
ORDER BY rank DESC
LIMIT 3;
```

The `cancellation` document should come first. Your rows and rank values depend on your seeded text. Try a query without `or` (`'cancel shipment fee'`: every word must match), an `or` query (`'insurance or damage'`) and an Arabic query (`'إلغاء'`: the `english` configuration does not stem Arabic, so expect weak or no results). Then check that the index is used:

```
EXPLAIN SELECT slug FROM policy_documents
WHERE search_vector @@ websearch_to_tsquery('english', 'insurance');
```

With only five rows PostgreSQL may choose a sequential scan; that is fine. With thousands of rows you would see `Bitmap Index Scan on policy_documents_search_vector_index`.

Notice `websearch_to_tsquery`: it accepts free text from users safely (quotes, `or`, `-word`) and never throws a syntax error, unlike `to_tsquery`.

Step 6 — The six tools (25 min)

What and why. Tools are the only way the agent can *do* anything. Each tool is a narrow, typed function the model can request. The model chooses *which* tool and *which arguments*; your PHP decides *whether it is allowed*.

Tool	Purpose	Rule
`get_quote`	Wraps `PricingService`, creates a `Quote`	City codes and vehicle type validated in PHP
`book_shipment`	Creates a shipment from a quote	Only if `confirmed: true`; quote must belong to the user and be unexpired and unused
`track_shipment`	Status and events by reference	Only the user's own shipments
`cancel_shipment`	Cancels by reference	Same `canTransitionTo` rules as the API
`search_policies`	Full-text search over `PolicyDocument`	Top 3 results
`escalate_to_human`	Creates a `SupportTicket`	Returns a ticket id such as `TKT-1`

AGENT PROMPT

Agent prompt

```
Plan first: list each tool class, its parameters and its return shape, then
wait for approval. Build tools one or two at a time, with tests, small diffs.

Create one class per tool in App\Agent\Tools: GetQuoteTool (get_quote),
BookShipmentTool (book_shipment), TrackShipmentTool (track_shipment),
CancelShipmentTool (cancel_shipment), SearchPoliciesTool (search_policies),
EscalateToHumanTool (escalate_to_human). Each class extends
Prism\Prism\Tool, takes the authenticated App\Models\User in its
constructor (SearchPoliciesTool needs no user), configures itself there with
$this->as(...)->for(...)->withStringParameter / withNumberParameter /
withEnumParameter / withBooleanParameter(...), and does its work in
__invoke(). Read vendor/prism-php/prism/src/Tool.php first and use only
methods that exist there.

Parameters:
- get_quote: origin_city_code, destination_city_code (enums of the city
  codes), vehicle_type (enum of VehicleType values), weight_kg.
- book_shipment: quote_id, pickup_at (ISO 8601), confirmed (boolean).
- track_shipment and cancel_shipment: reference. search_policies: query.
  escalate_to_human: reason.

Rules:
- The user always comes from the constructor. No tool takes a user_id
  parameter.
- Reuse existing code: PricingService::createQuote for get_quote;
  BookingService::book for book_shipment; BookingService::cancel (which uses
  the same canTransitionTo rules as the API) for cancel_shipment.
- book_shipment: if `confirmed` is not true, return a message telling the
  model to ask the customer to confirm, and create nothing.
- track_shipment and cancel_shipment look the shipment up through
  $user->shipments() and return "No shipment with that reference was found
  for this customer." otherwise (do not reveal that it exists).
- search_policies uses a PolicyDocument::search() scope: on PostgreSQL
  websearch_to_tsquery('english', ?) with a binding, ordered by ts_rank; on
  SQLite (tests) a LIKE fallback. Limit 3. Return the titles and bodies as
  text.
- escalate_to_human creates a SupportTicket row (user_id, reason, status
  open) and returns JSON with ticket_id "TKT-{id}".
- Tools return a short JSON string on success and a short plain-text
  message when they refuse or fail, so the model can explain it.

Tests in tests/Feature/Agent/ToolsTest.php: call each tool directly with
$tool->handle(...named args), the same way Prism calls it (no LLM).
Must include: book_shipment with confirmed=false creates no shipment;
track_shipment cannot see another user's shipment; cancel_shipment follows
the same rules as the API (it refuses an in_transit shipment); search_policies
returns the cancellation document for "cancel".
```

What to check.

- Good: the tests call the tool directly, for example `(new GetQuoteTool($user))->handle(origin_city_code: 'TIP', ...)`, and assert on the result and the database. A tool is just a PHP class; test it like one. (`get_quote` TIP → MRA, `box_truck`, 1000 kg must return `"price_lyd": "322.00"`.)
- Good: `PolicyDocument::search()` uses `whereRaw("search_vector @@ websearch_to_tsquery('english', ?)", [$tsQuery])` with a binding. Look for this yourself.
- Red flag: `"... websearch_to_tsquery('english', '$query')"` with string interpolation. That is SQL injection, written by the model, triggered by a customer message.
- Red flag: the booking logic is copy-pasted from the controller instead of reused. Two copies drift; one of them will forget the "quote already used" check.
- Red flag: a tool accepts `user_id` or `email` as a parameter. The model can be talked into passing anyone's id.

```
sail artisan test --filter=Tools
```

```
  Tests:    12 passed
```

(The reference `ToolsTest` has 12 tests. Your count may differ; every rule in the table above needs at least one test.)

Step 7 — ShahnaAssistant and conversation memory (20 min)

What and why. The agent loop: send the conversation plus tool definitions; if the model asks for a tool, run it and send the result back; repeat until the model answers in text, up to **6 steps**. Prism runs this loop for you when you call `withMaxSteps(config('shahna.max_steps'))`. Your job is the system prompt, the tool list and memory.

AGENT PROMPT

Agent prompt

```
Plan first and wait for approval. Small diffs, tests first.

1. Migrations and models AgentConversation (user_id, timestamps) and
   AgentMessage (conversation_id, role: user|assistant, content text,
   tool_calls json nullable, timestamps). Cast tool_calls to array. Add
   AgentMessage::toPrismMessage(). The support_tickets table from Step 6
   can live in the same migration.
2. App\Agent\ShahnaAssistant::reply(User $user, string $message,
   ?AgentConversation $conversation = null): array
   returning ['conversation_id' => int, 'reply' => string, 'actions' => array].
   - Load the previous messages of the conversation (the last
     config('shahna.history_limit')) and map them to Prism UserMessage /
     AssistantMessage objects (check the constructors in
     vendor/prism-php/prism/src/ValueObjects/Messages).
   - Prism::text()->using(Provider::Anthropic, config('shahna.agent_model')),
     withSystemPrompt, withMessages (history + new UserMessage), withTools
     (the six tools built for $user), withMaxSteps(config('shahna.max_steps')),
     then asText().
   - System prompt rules: answer in the customer's language (Arabic or
     English); use tools for prices, bookings, tracking and cancellations,
     never invent numbers; ALWAYS show the quote and ask for explicit
     confirmation before calling book_shipment with confirmed=true; for
     policy questions call search_policies and answer only from its results;
     escalate_to_human when unsure or when the customer asks for a person.
   - actions = one {tool, arguments} item per tool call, from all steps
     ($response->steps, each Step's toolCalls; ToolCall->name and
     ToolCall->arguments()).
   - If the final text is empty (max steps reached), reply with a fixed
     fallback message.
   - Persist the user message and the assistant reply (with tool_calls).
3. Put the system prompt in a private systemPrompt(User $user) method or a
   Blade view, your choice, and tell me why.

Tests (tests/Feature/Agent/AgentChatTest.php, through the endpoint from
Step 8) with Prism::fake() and TextResponseFake: a new conversation is
created and both messages stored; a second call with the same conversation
sends the earlier messages, the system prompt, max steps and the six tools to
the model (assert with $fake->assertRequest(...)); a two-step fake built with
ResponseBuilder and TextStepFake (a get_quote tool call, then text) shows up
in actions.
```

What to check.

- Good: a test asserts the history is actually sent. Memory bugs are silent: the agent “works” but forgets everything.
- Good: `withMaxSteps(config('shahna.max_steps'))` is present. Without it Prism makes a single step and tool results never reach a final answer.
- Red flag: the conversation is loaded with `AgentConversation::find($id)` and no ownership check. Any user could read any conversation by guessing ids. It must be `$user`

`->agentConversations()->findOrFail($id)` or equivalent.

- Red flag: the whole conversation is sent forever. Twenty messages (`shahna.history_limit`) is a deliberate limit; ask the agent where it is enforced.

> **TIP**
>
> `Prism::fake()` does not execute your tools: it simply returns the fake responses in order. A multi-step fake built with `ResponseBuilder` and `TextStepFake` only *describes* tool calls and results. That is why Step 6 tests tools directly and Step 7 tests the wiring (prompt, history, persistence, `actions`). Together they cover the agent without a single real API call.

Step 8 — The chat endpoint and manual tests (15 min)

AGENT PROMPT

Agent prompt

```
Plan first. Tests first. Small diff.

Add POST /api/agent/chat behind auth:sanctum.
- FormRequest AgentChatRequest: message required string max 2000;
  conversation_id nullable integer.
- Invokable AgentChatController: load the conversation with
  $request->user()->agentConversations()->findOrFail(...) (someone else's
  conversation is a 404), call ShahnaAssistant and return 200 with
  {conversation_id, reply, actions}.
Feature tests in tests/Feature/Agent/AgentChatTest.php with Prism::fake():
422 without message; 200 with the right JSON shape; 404 when
conversation_id belongs to another user.
Then run the full test suite.
```

Now test it for real. Log in as a customer and keep the token:

```
TOKEN=$(curl -s -X POST http://localhost/api/auth/login \
  -H "Accept: application/json" -H "Content-Type: application/json" \
  -d '{"email":"customer@shahna.test","password":"password"}' | jq -r .token)
```

(`customer@shahna.test` / `password` is the seeded demo customer.)

Arabic price question:

```
curl -s -X POST http://localhost/api/agent/chat \
  -H "Authorization: Bearer $TOKEN" \
  -H "Accept: application/json" -H "Content-Type: application/json" \
  -d '{"message":"كم سعر شحن 800 كيلو من طرابلس إلى بنغازي بشاحنة صندوقية؟"}' | jq
```

```
{
  "conversation_id": 1,
  "reply": "سعر شحن 800 كيلوغرام من طرابلس إلى بنغازي بشاحنة صندوقية هو 1290.00 دينار ليبي (رقم العرض 31).
        العرض صالح لمدة 15 دقيقة. هل تريد تأكيد الحجز؟",
  "actions": [
    {"tool": "get_quote", "arguments": {"origin_city_code": "TIP", "destination_city_code":
        "BEN", "vehicle_type": "box_truck", "weight_kg": 800}}
  ]
}
```

The reply text and the quote ID will differ. Check the price **by hand** with the pricing formula and the seeded distance (Tripoli to Benghazi is 1020 km): `50 + 1020 × 1.2 + 800 × 0.02 = 1290.00`. If the reply's number does not match `get_quote`'s result, the model is inventing prices, and your system prompt needs work.

The confirmation rule. Continue the same conversation, but ask to book without confirming details:

```
curl -s -X POST http://localhost/api/agent/chat \
  -H "Authorization: Bearer $TOKEN" \
  -H "Accept: application/json" -H "Content-Type: application/json" \
  -d '{"message":"Book it for tomorrow 9am","conversation_id":1}' | jq '.reply, .actions'
```

Good: the reply restates the price and pickup time and asks "Shall I confirm this booking?"; `actions` contains no `book_shipment` with `confirmed: true`. Then send `{"message":"Yes, confirm","conversation_id":1}` and check that a shipment now exists:

```
sail artisan tinker --execute="dump(App\Models\Shipment::latest()->first()?
       ->only('reference','status','price_lyd'));"
```

English policy question:

```
curl -s -X POST http://localhost/api/agent/chat \
  -H "Authorization: Bearer $TOKEN" \
  -H "Accept: application/json" -H "Content-Type: application/json" \
  -d '{"message":"Can I cancel after the truck picked up my goods?"}' | jq '.reply, .actions'
```

Expected: `actions` includes `search_policies`, and the reply says cancellation is not possible after pickup. Compare it word by word with your seeded policy.

Finally, make sure the confirmation behaviour is locked in by a test. Because `Prism::fake()` does not run tools, this test belongs at the tool level: `ToolsTest` must call `book_shipment` with `confirmed: false` and assert that no shipment is created.

CHECKPOINT

Checkpoint 2 (15:45)

```
sail artisan test
```

```
   PASS  Tests\Feature\Agent\AgentChatTest
   PASS  Tests\Feature\Agent\IntentClassifierTest
   PASS  Tests\Feature\Agent\ShipmentEmailExtractorTest
   PASS  Tests\Feature\Agent\ToolsTest
   ...
  Tests:    N passed
```

Zero failures, zero real API calls in the suite. The `day4-end` reference has 104 tests (80 from Day 3 plus 24 new ones).

Quick API checks:

```
curl -s -X POST http://localhost/api/agent/chat -H "Accept: application/json" \
  -H "Content-Type: application/json" -d '{"message":"hi"}'
```

```
{"message":"Unauthenticated."}
```

```
curl -s -X POST http://localhost/api/agent/chat -H "Authorization: Bearer $TOKEN" \
  -H "Accept: application/json" -H "Content-Type: application/json" -d '{}'
```

```
{"message":"The message field is required.","errors":{"message":["The message field is
        required."]}}
```

```
git add -A
git commit -m "feat: shahna assistant with tools, retrieval and memory"
```

If you're stuck

The reference solution is the `day4-end` tag:

```
git diff --stat day4-end
git show day4-end:app/Agent/ShahnaAssistant.php
git diff day4-end -- app/Agent/Tools/
git show day4-end:database/seeders/PolicyDocumentSeeder.php
```

Most common blockers and where to look:

Symptom	Likely cause
`Class "Prism\Prism\..." not found`	The agent invented a class. `ls vendor/prism-php/prism/src` and search for it.
Agent replies but never uses tools	`withTools()` missing, or `withMaxSteps()` left at 1
Tests are slow or need the network	A test is missing `Prism::fake()`
Arabic search returns nothing	Expected: the `'english'` configuration does not understand Arabic. The tool asks the model for English keywords
Every test fails with a SQL error about `tsvector`	The PostgreSQL-only migration SQL is not guarded with `DB::getDriverName() === 'pgsql'`
`401` from Anthropic in Tinker	`ANTHROPIC_API_KEY` missing; run `sail artisan config:clear` after editing `.env`

To reset your branch to the reference (your work stays on `my-day4`):

```
git stash
git switch -c my-day4-reset day4-end
sail composer install
sail artisan migrate:fresh --seed
sail artisan test
```

Stretch goals

1. **Route by intent.** Call `IntentClassifier` first in the chat controller. For `policy` intent, call `search_policies` directly and answer with Haiku and no other tools. Measure how many Sonnet calls you saved over 10 messages. (This becomes required tomorrow.)
2. **Email intake endpoint.** Add `POST /api/quotes/from-email` that runs `ShipmentEmailExtractor`, returns `422` listing the missing fields when anything is `null`, and otherwise creates a quote through `PricingService`.
3. **pgvector.** Add the `pgvector` extension, store embeddings for the policy documents, and compare the top-3 results with full-text search for 10 questions. Which questions does each method get wrong?
4. **Arabic search.** Check `SELECT cfgname FROM pg_ts_config;` for an `arabic` configuration. Add Arabic text to a policy, try `'simple'` and `arabic` configurations, and compare results for `'إلغاء'` vs `'الإلغاء'`.

Reflection

1. Which Prism class or method did the coding agent get wrong today, and how did you find out (error, `grep` in `vendor/`, Boost docs search)? What would have happened if the wrong code had happened to run?
2. List every place where *your PHP code*, not the model, enforces a rule (ownership, confirmation, enum values, SQL bindings). Which of these would break if you trusted the model's arguments?
3. Your tests all use fakes. What *can't* they tell you about the real assistant, and how would you find out? (Hint: that is tomorrow's `agent:eval`.)